

s4.02 — Chat system — durable delivery and offline users

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Durable send and per-conversation sequence — IC3 baseline](#)
- [Delivery, offline catch-up, fan-out, and presence — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: first-hand `1p3a` prompt retained in [the bank](#).
 - Prompt as reported: chat system with durable delivery and offline users.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: a WebSocket server relays each message from sender to recipient and replies "sent".
 - First break: the recipient is offline, or the gateway crashes between receiving and relaying, and the message is gone
 - "Sent" was a transport event, not a durable delivery promise.
 - Second break: the sender's acknowledgment is lost, the client retries, and the recipient sees the message twice
 - At-least-once transport without a stable client message identity creates duplicates.
 - Third break: two members send at once, each device orders by its own clock, and the conversation reads differently on every screen
 - Client timestamps cannot define order.
 - Fourth break: a user with `50` conversations comes back after a day and the client asks "what did I miss?" and the answer costs `50` queries
 - Catch-up needs an index, not a scan.
 - Minimum primitives
 - A durable append per conversation with a server-assigned sequence
 - A sender intent key that makes retries idempotent
 - Per-device cursors that advance only over contiguous sequences
 - A route registry that separates "who is connected where" from "what is durably owed"
 - An inbox index that lists which conversations have sequences beyond a device's cursor

- 🗨️ “Delivery is a cursor over a durable per-conversation log, not a WebSocket frame; presence only decides whether to push now or wait.”
- Opening script
 - “I will treat this as a durable per-conversation log with server-assigned sequences, and delivery as per-device cursors over that log.”
 - “First I confirm group sizes, the ordering scope, what a sender's acknowledgment must mean, and how long an offline user can be away.”
 - “Then I size sends, concurrent connections, and fan-out, which tells me receipts and large groups dominate the write load, not messages.”
 - “The architecture is a stateless gateway fleet, a message service that commits message plus outbox in one transaction, delivery workers, a route registry for presence, and an inbox index for catch-up.”
 - “The deep dive is the send transaction that allocates the sequence and deduplicates the sender's intent, and the reconnect path that resumes from cursors without loss or duplication.”
 - “I will close with the failure table, membership-change safety, and where the design changes for very large groups and ten times the load.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Send rate, connections, fan-out multiplier; API semantics
Architecture	17:00–30:00	Diagram, gateway/message/delivery split, one send trace
Deep dive	30:00–45:00	Send transaction, sequence, cursors, reconnect catch-up
Operations and tradeoffs	45:00–50:00	Failure table, membership safety, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Direct messages only, or groups?	Both; groups up to 256 members by default, larger channels as a stretch	Fan-out on write is affordable at 256; large channels need a pull model
What does the sender's acknowledgment mean?	The message is durably committed and will be delivered to every current member at least once	Acknowledge after commit, not after relay; delivery is tracked per device
Ordering scope?	Total order within one conversation; no cross-conversation order	One sequence per conversation; no global clock
How long can a user be offline?	Up to 30 days; history beyond that is fetched on demand	Inbox index plus durable history; cursors are the only client state that matters
Multiple devices per user?	Yes, up to 5	Cursors are per device; a read on one device advances the read receipt for the user
Do members change during a conversation?	Yes, join and leave are common	Visibility is bounded by <code>joined_sequence</code> and <code>left_sequence</code> ; delivery checks membership at delivery time
Attachments?	By reference; bytes uploaded separately	Message payloads stay small; the upload service is a dependency, not part of this design

- Functional scope

- Create conversations and manage membership.
- Send a message and receive a durable acknowledgment with a sequence.
- Deliver to every online device now and every offline device on reconnect.
- Report delivered and read receipts per device.
- Resume a device from its cursors without loss or duplication.
- Show approximate presence.

Requirement	Target	Enforced by
Durability after acknowledgment	Zero loss; survives one replica and one process failure	Synchronous replicated commit before 201; outbox in the same transaction
Duplicate suppression	0 duplicate messages per (conversation, sender, clientId)	Unique key in the message table; retry returns the prior messageId
Ordering	Every device renders one conversation in the same order	Server-assigned contiguous sequence; clients render by sequence only
Online delivery latency	p95 under 500 ms from commit to recipient device	Outbox tailed within 100 ms; route lookup from an in-memory registry
Offline catch-up	A device off for 30 days with 50 active conversations syncs in under 5 s	Inbox index keyed by user; one bounded page per conversation
Send latency	p99 under 300 ms at the API	One transaction on one shard; no fan-out inside the send path
Concurrent connections	2 M per region	Gateway fleet at 50 k connections per node with 50% headroom
Presence accuracy	Online state within 90 s of a disconnect	Heartbeat every 30 s; registry entry TTL 90 s
Availability of send	99.95% monthly	Stateless gateway and message tiers; sharded replicated store

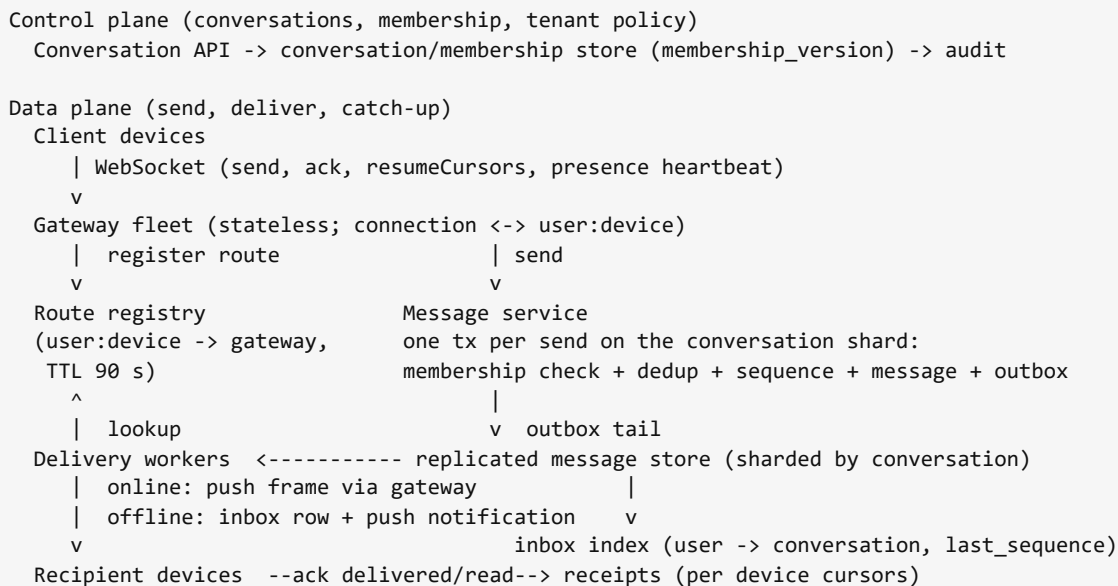
- Invariants
 - Tenant and membership authorization apply to send, read, receipts, and delivery.
 - A conversation's sequences are contiguous and assigned only inside the send transaction.
 - A message with a given (conversation, sender, clientId) is committed at most once.
 - A device's delivered_through advances only over a contiguous prefix it has actually received.
 - read_through never exceeds delivered_through.
 - Transport delivery is never the durability record; the store is.
 - A removed member receives no message with a sequence above its left_sequence.
- Non-goals
 - End-to-end encryption key exchange.
 - Message search and full-text indexing.
 - Voice, video, or live typing indicators beyond presence.
 - Cross-tenant federation.

Capacity and consequence — IC3 baseline

- Sends
 - Assume 10 M daily active users sending 40 messages per day.
 - That is 400 M messages per day, about 4,600/s average and about 23,000/s at a 5× peak.
 - Decision: one transaction per send on a conversation-sharded store
 - 23,000/s across 64 shards is about 360 writes/s per shard, comfortable for a replicated SQL primary.
- Storage
 - Assume 1 KB per stored message including indexes and metadata.
 - 400 M × 1 KB = 400 GB/day, about 146 TB per year before compression.
 - Decision: partition message tables by conversation and by month

- Hot history is the last 90 days; older partitions move to cheaper storage but stay queryable.
- Fan-out
 - Assume an average of 3 recipient devices per message across direct and small-group traffic.
 - Peak delivery events are about 70,000/s; a 256-member group multiplies one send into 256 pushes.
 - Decision: fan-out on write up to 256 members, fan-out on read above that
 - Large channels keep one timeline; members pull by sequence and receive only a "new sequence" hint.
- Receipts
 - Two receipts per delivery event would be about 140,000/s at peak, more than sends.
 - Decision: clients coalesce receipts per (conversation, device) with a 2 s debounce and send only the highest contiguous sequence
 - Receipt writes drop by roughly an order of magnitude and stay idempotent because cursors are monotonic.
- Connections
 - Assume 20% of daily users connected at once, so 2 M concurrent connections per region.
 - At 50 k connections per gateway node that is 40 nodes; run 60 for 50% headroom across fault domains.
 - Decision: gateways hold no durable state
 - A gateway crash drops 50 k connections whose clients reconnect and resume from cursors; nothing is lost.
- Catch-up
 - A user with 50 conversations reconnecting after a day must not issue 50 history queries.
 - Decision: keep a per-user inbox index of (conversation, last_sequence)
 - One indexed read lists the conversations with new sequences; only those are paged.

Architecture and flow — IC3 baseline



- Gateway fleet owns connections.
 - Mechanism bought: one long-lived connection per device, multiplexing sends, acks, and pushes.
 - Cost: 2 M connections of state that must be rebuildable, so gateways store nothing durable.
- Message service owns the send transaction.
 - Mechanism bought: membership check, sender dedup, sequence allocation, message insert, and outbox rows commit atomically on one shard.
 - Cost: one row per conversation is a serialization point; a very hot conversation contends on it.
- Replicated message store owns durability and order.

- Mechanism bought: synchronous replication before acknowledgment; sequence-indexed history for catch-up.
 - Cost: send latency includes replica round trips; the store is the availability floor for send.
 - Delivery workers own at-least-once push.
 - Mechanism bought: tail the outbox, look up routes, push to online devices, write inbox rows for offline devices.
 - Cost: pushes can repeat after a worker crash, so clients must deduplicate by sequence.
 - Route registry owns presence and routing.
 - Mechanism bought: `user:device` to gateway mapping with a TTL; approximate online state.
 - Cost: it is stale by up to `90 s` and never owns durability; a wrong route only delays delivery.
 - Inbox index owns catch-up.
 - Mechanism bought: one read per user lists conversations with unseen sequences.
 - Cost: one extra write per recipient per message in small groups; a hint-only write in large channels.
 - Control plane owns conversations, membership, and tenant policy.
 - Mechanism bought: membership changes carry a `membership_version` that delivery checks.
 - Cost: cached membership can be stale, so delivery must verify against the version at push time.
 - Static stability
 - Sends, pushes, and catch-up continue on the message store and route registry while the conversation control plane is unavailable.
 - Creating conversations and changing membership wait for the control plane.
 - Blast radius
 - Conversation shards, gateway nodes, and delivery workers are spread across fault domains.
 - A shard failure affects only its conversations; a gateway failure affects only its connections.
- Device sends `{clientId, body, attachmentRefs}` over its WebSocket to the gateway.
- Gateway forwards to the message service with the authenticated sender and device.
- Message service opens one transaction on the conversation's shard.
- It verifies membership at the current `membership_version` and checks `(conversation, sender, clientId)`.
- It increments `last_sequence`, inserts the message at that sequence, and inserts outbox rows for current members.
- The transaction commits with synchronous replication and the gateway returns `{messageId, sequence}` to the sender.
- A delivery worker tails the outbox, looks up each member device in the route registry, and pushes the frame through that device's gateway.
- Offline devices get an inbox row and a content-free push notification.
- Each receiving device acknowledges `deliveredThrough` after rendering a contiguous prefix; the server stores the cursor.

API and data model — IC3 baseline

- The detailed contract lives in the [chat companion](#); this table restates it with idempotency and conflict behavior.

Surface	Contract	Conflict/error behavior
POST <code>/conversations</code>	<code>{type, members}</code> ; returns 201 conversation	Direct conversation for the same pair returns the existing one; policy violations return 403
POST <code>/conversations/{id}/messages</code>	<code>{clientId, body, attachmentRefs}</code> ; returns 201 <code>{messageId, sequence}</code> after durable commit	Same <code>clientId</code> returns the prior result; non-member returns 404; oversized body returns 413

Surface	Contract	Conflict/error behavior
GET /conversations/{id}/messages? afterSeq=&limit=	{items, nextSeq} in ascending sequence	Bounded by joined_sequence and left_sequence for the caller; 404 across tenants
PUT /conversations/{id}/receipts/{device}	{deliveredThrough, readThrough}; returns 200 receipt	Non-monotonic or readThrough > deliveredThrough returns 400; device owner only
PUT /conversations/{id}/members/{user}	{role}; returns 200 membership with new membership_version	Stale version precondition returns 409; administrator and tenant boundary
DELETE /conversations/{id}/members/{user}	Returns 204; sets left_sequence	Idempotent; removal invalidates future delivery authorization
WebSocket	{resumeCursors} on connect; send, ack, presence frames	Same send dedup as REST; reconnect replays from durable history, never from gateway memory

State	Key	Fields/invariant
Conversation	(tenant_id, conversation_id)	Kind, last_sequence, membership_version; the sequence counter lives here
Membership	(tenant_id, conversation_id, user_id)	Role, joined_sequence, left_sequence, version; index by (tenant, user) for the conversation list
Message	(tenant_id, conversation_id, sequence)	message_id, sender, client_message_id, payload ref, created time, tombstone; unique (conversation, sender, client_message_id) and unique (conversation, message_id)
Outbox	(tenant_id, conversation_id, sequence, user_id)	Written in the send transaction; deleted after delivery attempt is recorded
Receipt	(tenant_id, conversation_id, user_id, device_id)	delivered_through, read_through; check read_through <= delivered_through
Inbox index	(tenant_id, user_id, conversation_id)	last_sequence seen by the conversation; drives catch-up
Route	presence: {tenant:user:device}	connectionRoute, expiresAt; approximate, never owns durability

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - The route registry lives in an in-memory store with a 90 s TTL and message bodies live behind payload_ref; conversations, membership, messages, outbox, receipts, and the inbox index are relational.

```

CREATE TABLE conversations (
  tenant_id text NOT NULL, conversation_id text NOT NULL, kind text NOT NULL,
  last_sequence bigint NOT NULL DEFAULT 0, -- the sequence counter; incremented in
the send transaction
  membership_version bigint NOT NULL DEFAULT 0,
  PRIMARY KEY (tenant_id, conversation_id),
  CHECK (kind IN ('DIRECT', 'GROUP', 'CHANNEL')),
  CHECK (last_sequence >= 0)
);
CREATE TABLE conversation_members (
  tenant_id text NOT NULL, conversation_id text NOT NULL, user_id text NOT NULL,
  role text NOT NULL, joined_sequence bigint NOT NULL, left_sequence bigint,
  version bigint NOT NULL, -- PUT precondition
  PRIMARY KEY (tenant_id, conversation_id, user_id),
  FOREIGN KEY (tenant_id, conversation_id) REFERENCES conversations,
  CHECK (left_sequence IS NULL OR left_sequence >= joined_sequence)
);
CREATE INDEX memberships_user ON conversation_members (tenant_id, user_id, conversation_id);
CREATE TABLE chat_messages (
  tenant_id text NOT NULL, conversation_id text NOT NULL, sequence bigint NOT NULL,
  message_id text NOT NULL, sender_id text NOT NULL, client_message_id text NOT NULL,
  payload_ref text NOT NULL, created_at timestamptz NOT NULL,
  tombstone boolean NOT NULL DEFAULT false, -- keeps its sequence so cursors stay
contiguous
  PRIMARY KEY (tenant_id, conversation_id, sequence),
  UNIQUE (tenant_id, conversation_id, sender_id, client_message_id),
  UNIQUE (tenant_id, conversation_id, message_id),
  FOREIGN KEY (tenant_id, conversation_id) REFERENCES conversations,
  FOREIGN KEY (tenant_id, conversation_id, sender_id) REFERENCES conversation_members,
  CHECK (sequence > 0)
);
CREATE TABLE chat_outbox (
  tenant_id text NOT NULL, conversation_id text NOT NULL, sequence bigint NOT NULL,
  user_id text NOT NULL,
  lease_until timestamptz, -- NULL = unclaimed; worker lease for the
delivery attempt
  attempts integer NOT NULL DEFAULT 0,
  PRIMARY KEY (tenant_id, conversation_id, sequence, user_id),
  FOREIGN KEY (tenant_id, conversation_id, sequence) REFERENCES chat_messages,
  CHECK (attempts >= 0)
);
CREATE INDEX outbox_expired_leases ON chat_outbox (lease_until) WHERE lease_until IS NOT NULL;
CREATE TABLE chat_receipts (
  tenant_id text NOT NULL, conversation_id text NOT NULL, user_id text NOT NULL,
  device_id text NOT NULL,
  delivered_through bigint NOT NULL DEFAULT 0, read_through bigint NOT NULL DEFAULT 0,
  PRIMARY KEY (tenant_id, conversation_id, user_id, device_id),
  FOREIGN KEY (tenant_id, conversation_id, user_id) REFERENCES conversation_members,
  CHECK (read_through <= delivered_through),
  CHECK (delivered_through >= 0)
);
CREATE TABLE chat_inbox (
  tenant_id text NOT NULL, user_id text NOT NULL, conversation_id text NOT NULL,
  last_sequence bigint NOT NULL, -- upserted by delivery workers, outside
the send transaction
  PRIMARY KEY (tenant_id, user_id, conversation_id),
  FOREIGN KEY (tenant_id, conversation_id, user_id) REFERENCES conversation_members,
  CHECK (last_sequence > 0)
);

```

- What the DDL does not capture
 - The Route row is presence:{tenant:user:device} → connectionRoute, expiresAt in the route registry; it is never relational and never owns durability.
 - Message bodies and attachments live behind payload_ref and the upload service; SQL holds the order, identity, and cursors only.

- Deviation from the companion: `chat_outbox` and `chat_inbox` are added because the state table lists them, and `chat_outbox` is what the send transaction writes; `chat_inbox` is eventually consistent and can move to a key-value store without changing the contract.
- Conventions
 - The sender is inferred from authentication; the request never names it.
 - `messageId` is a server identifier; `sequence` is the order; `clientId` is the retry key and is never shown to recipients.
 - Tombstones keep their sequence so deletion does not create gaps that break contiguous cursors.
 - Attachments are uploaded through the [resumable upload service](#) and referenced by an authorized handle before publication.
 - Shared conventions follow the [common API rules](#).

Durable send and per-conversation sequence — IC3 baseline

- Why the sequence lives in the conversation row
 - A contiguous per-conversation sequence gives every device the same order and lets cursors be a single integer.
 - Allocating it inside the same transaction as the insert means no gaps and no two messages with the same sequence.
 - A global ID generator would give uniqueness but not contiguity, and cursors need contiguity to detect gaps.
- Send transaction

```
SEND(tenant, conv, sender, device, clientId, body):
  tx on shard(conv):
    m = membership(conv, sender) FOR SHARE
    if m is null or m.left_sequence is not null: abort NOT_MEMBER
    prior = message_by_intent(conv, sender, clientId)
    if prior exists: return {prior.message_id, prior.sequence} # idempotent retry
    c = conversation(conv) FOR UPDATE # serializes senders per conversation
    seq = c.last_sequence + 1
    insert message(conv, seq, new message_id, sender, clientId, payload_ref)
    for member in current_members(conv): insert outbox(conv, seq, member)
    update conversation set last_sequence = seq
    commit with synchronous replication
  return {message_id, seq}
```

- Concurrency
 - `FOR UPDATE` on the conversation row serializes concurrent senders in the same conversation; different conversations never contend.
 - The membership read is inside the transaction so a removal that commits first makes the send fail with `NOT_MEMBER`.
 - The dedup lookup is inside the transaction so two retries of the same intent cannot both insert; the unique key is the backstop.
- Consistency boundary
 - Message, outbox, and sequence commit together on one shard, so a crash leaves either nothing or a complete send.
 - Delivery, receipts, and the inbox index are outside the transaction and are eventually consistent by design.
 - The acknowledgment to the sender is sent only after commit, so "sent" means "durable and owed to every current member".
- Lost acknowledgment
 - The client keeps `clientId` and retries after a timeout.
 - The retry returns the same `messageId` and `sequence`; no second row is written.

- A client that generates a new `clientId` per retry defeats this, so the client contract requires one id per user intent.
- Hot conversation
 - At 360 writes/s per shard the row lock is fine; one conversation at 1,000 sends/s would serialize on its row and add queueing latency.
 - Baseline mitigation: rate-limit sends per conversation and keep the transaction short by writing outbox rows in bulk.
 - The structural fix is IC4 stretch: move sequence ownership to an in-memory partition leader that batches allocations and appends to the log.
- Exact failure this prevents
 - A sender's connection drops after the server commits but before the acknowledgment arrives.
 - Without the intent key the retry commits a second message; recipients see a duplicate and cursors still advance.
 - With the intent key the retry returns the original sequence and the recipient sees one message.

Delivery, offline catch-up, fan-out, and presence — IC3 baseline

- Delivery worker loop
 - Workers tail the outbox in sequence order per conversation and claim rows with a lease.
 - For each (conversation, sequence, user) they enumerate the user's devices and look up routes.
 - Online device: push the frame through its gateway; the gateway forwards; the worker records an attempt.
 - Offline device: upsert the inbox row (user, conversation) → last_sequence and enqueue a content-free push notification.
 - The outbox row is deleted after the attempt is recorded; a crash before that repeats the attempt.
- At-least-once and client dedup
 - A repeated push carries the same sequence, so the client discards anything at or below its rendered prefix.
 - delivered_through moves only when the client has a contiguous prefix; a gap triggers a GET ...?afterSeq= fill.
- Receipts
 - Delivered and read cursors are per device; the user-level read position is the maximum over devices.
 - Cursors are monotonic, so a late or repeated receipt is a no-op rather than a regression.
 - Read receipts fan out to other members as lightweight events; they are approximate and are not part of durability.
- Reconnect and offline catch-up
 - Device reconnects and sends resumeCursors as {conversation: deliveredThrough} for conversations it knows.
 - Gateway registers the route and asks the inbox index for conversations where last_sequence > deliveredThrough.
 - Server returns those conversations with their new sequence ranges.
 - Device pages each range with GET ...?afterSeq=&limit= from durable history, in ascending sequence.
 - Device advances deliveredThrough per conversation as each contiguous page renders.
 - Any push that arrives during catch-up is deduplicated by sequence and does not advance the cursor past a gap.
- Fan-out policy
 - Small conversations (up to 256 members): outbox rows and inbox writes per member at send time, so catch-up is one indexed read.
 - Large channels: one timeline; the send writes a single channel-level last_sequence, and online members receive a hint to pull.
 - Members of a large channel catch up by reading the channel timeline from their cursor; no per-member rows are written per message.

- The policy is chosen per conversation by member count and is stable for the conversation's lifetime unless migrated explicitly.
- Membership changes
 - `joined_sequence` and `left_sequence` bound what a member can read; history visibility policy decides whether a joiner sees earlier sequences.
 - Delivery workers check membership at push time against the current `membership_version`, not a cached list from send time.
 - A removed member's outbox rows are dropped; a message committed after removal never reaches that member.
 - Reconnect re-validates membership before serving catch-up pages.
- Presence
 - Devices heartbeat every `30 s`; the route entry expires after `90 s` without one.
 - Presence is approximate and only decides push-now versus inbox-plus-notification.
 - A stale route causes a failed push, which the worker treats as offline; nothing is lost because the inbox row is written on that path.
 - Presence fan-out to contacts is rate-limited and coalesced; it never blocks message delivery.
- Push notifications
 - Notifications carry conversation id and a counter, not message content, so they can be lost or delayed safely.
 - The app fetches content over the authenticated channel on open.
- Exact failure this prevents
 - A gateway node crashes with `50 k` connections while `2,000` messages are in flight to those devices.
 - Without the outbox and cursors those messages are gone from memory and the recipients never learn of them.
 - With the outbox, the worker's next attempt finds no route, writes inbox rows, and the devices catch up from their cursors on reconnect.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Sender acknowledgment lost	One message	Client timeout	Client retries with the same intent id	Dedup returns the prior sequence
Gateway node crash	Its <code>50 k</code> connections	Health check, connection drop	Clients reconnect to another node with cursors	Catch-up from durable history; no loss
Message shard primary down	That shard's conversations	Replication health, send errors	Sends to those conversations return <code>503</code> ; reads continue on replicas	Promote replica; sends resume with the same sequence counter
Delivery worker crash	Its leased outbox rows	Lease expiry	Delivery delayed; pushes may repeat	Another worker reclaims the lease; clients dedup by sequence
Route registry unavailable	All online pushes	Lookup errors	Treat every device as offline: inbox rows plus notifications	Registry restored; gateways re-register routes
Push notification provider down	Offline users' wake-ups	Provider errors	Users see messages on next open	Notifications are not retried past a short window
Membership cached stale	One conversation's removed member	Version mismatch at push	Push is dropped; message stays in history	Worker re-reads membership at the current version
Hot conversation	One conversation's senders	Row lock wait time	Per-conversation send rate limit; others unaffected	Move sequence ownership to a partition leader

Failure	Blast radius	Detection	Degraded behavior	Recovery
Control-plane conversation store unavailable	New conversations and membership changes	Health check, error rate	Sends, pushes, and catch-up continue at the last membership version	Fail over replica; queued membership changes apply in version order
Cross-region link down	Users homed in the other region	Replication lag alarm	Each conversation serves from its home region; remote users see delay	Resume replication; sequences are per conversation so no conflict

- Observe
 - Commit-to-push latency p95 per region.
 - Outbox age and lease reclaim rate.
 - Catch-up page count and duration per reconnect.
 - Dedup hit rate on sends, which measures retry health.
 - Route lookup miss rate and stale-route push failures.
 - Per-conversation lock wait time for the hottest conversations.
- Retry discipline
 - Clients retry sends with exponential backoff and jitter, reusing `clientMessageId`.
 - Delivery workers retry pushes a bounded number of times before falling back to the offline path.
 - Receipts are never retried aggressively; the next receipt supersedes a lost one because cursors are monotonic.
 - A `NOT_MEMBER` or authorization failure is never retried; the client refreshes its conversation list.
- Tenant isolation
 - Conversation ids, routes, and inbox rows are keyed by tenant; a token from another tenant receives `404`, not `403`, to avoid existence leaks.
 - Per-tenant quotas cap sends per second, group size, and connections so one tenant's storm cannot starve the gateway fleet.
 - Large-channel fan-out is capped per tenant so a single channel cannot consume the delivery fleet.
- Rollout and rollback safety
 - Gateway deploys drain connections gradually; clients reconnect with cursors so a deploy costs a reconnect, not messages.
 - Message schema changes follow expand, migrate, contract; a new delivery worker must read outbox rows written by the previous version.
 - Canary one shard's delivery workers and compare commit-to-push latency and dedup rates before fleet rollout.
 - Rollback never touches sequences; a rolled-back worker version simply resumes the outbox from its lease.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Relay-only WebSocket	Gateway forwards frames; store only for offline recipients	Lowest latency, least storage	Any crash between receive and forward loses messages; no shared order	Ephemeral signaling, never durable chat
Durable log plus cursors	Per-conversation sequence, outbox, per-device cursors	Loss-free, deduplicated, resumable	Storage and two extra writes per recipient	Baseline for durable delivery with offline users
Per-user mailbox only	Copy every message into each recipient's mailbox	Simple catch-up per user	Storage multiplies by members; order across members diverges	Very small groups with no shared timeline
Fan-out on read only	One timeline; every device polls each conversation	No per-recipient writes	Catch-up scans every conversation; wasteful for small groups	Large channels with many passive readers

Option	What it is	Strength	Weakness	Choose when
Message broker per conversation	A topic per conversation with consumer groups	Native ordering and replay	Millions of topics; per-device cursors are still needed	A managed log service with cheap topic creation

- IC4 stretch — stronger requirement: end-to-end encryption
 - The server stores ciphertext and never sees bodies; sequence, dedup, and cursors work unchanged on opaque payloads.
 - Server-side features that read content (search, moderation) move to the client or disappear.
 - Key distribution and device enrollment become a separate control-plane service with its own audit.
- IC4 stretch — very large channels
 - Above 256 members the send writes one timeline row and one channel-level `last_sequence`; members pull from cursors.
 - Online members receive a hint frame with the new sequence rather than the payload; the client pulls a page.
 - Read receipts collapse to per-user counters sampled periodically rather than per-message events.
- IC4 stretch — 10× load
 - 230,000 sends/s at peak and 20 M connections per region.
 - Move sequence allocation to an in-memory partition leader per shard that batches allocations and appends to a replicated log; the per-conversation contract is unchanged.
 - Cell the gateway and delivery fleets per tenant group so a storm in one cell cannot consume the rest.
 - Coalesce outbox rows per (conversation, sequence range) for small groups to cut delivery writes.
- IC4 stretch — cost and blast radius
 - Receipt and inbox writes dominate cost; debounce receipts on the client and coalesce inbox updates per (user, conversation).
 - Tier message partitions older than 90 days to cheaper storage with the same sequence index.
 - Place each conversation shard, its replicas, and the delivery workers that serve it across fault domains so one domain loss degrades latency, not durability.
- IC4 stretch — multi-region
 - Home each conversation in one region that owns its sequence; remote members connect locally and their sends are forwarded to the home region.
 - Sequences never conflict because only the home region allocates them; the cost is cross-region send latency for remote members.
- Weakest assumption
 - Clients honor the intent-id contract and the contiguous-cursor rule.
 - A client that mints a new `clientId` per retry or acknowledges past a gap creates duplicates or silent loss that the server cannot detect.
 - Mitigation: server-side sanity checks that reject a cursor advance beyond the highest pushed sequence, plus a dedup-rate alarm per client version.
- Over-engineering warning
 - Do not build a global message ordering service; ordering is per conversation and that is what the product needs.
 - Do not make presence strongly consistent; it is a routing hint and strong consistency buys nothing for durability.
 - Do not fan out on write for a 100,000-member channel.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Group size, ordering scope, acknowledgment meaning, offline horizon	Names multi-device and membership-change semantics as invariants	Asks what "sent" means before drawing anything
Estimates	Send rate, connections, fan-out multiplier, storage	Shows receipts outnumber sends and debounces them	A number changes the fan-out policy
Architecture	Gateway, message service, delivery, route registry, inbox split with one trace	States what still works when the control plane or registry is down	Every component has a mechanism and a price
Deep dive	Send transaction with sequence and dedup; cursors and reconnect catch-up	Hot-conversation contention and partition-leader sequencing; membership version at push time	Reasons across the store, outbox, and client boundary
Operations	Failure table, dedup and outbox observability	Cell blast radius, gateway drain on deploy, cost of receipts	Volunteers degradation and rollout before being asked
Wrap	Recap and tradeoff	Names client contract compliance as the flaw first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: The WebSocket write succeeded; is the message delivered?
 - **IC3 baseline**
 - No; transmission is not durable delivery, and only the recipient device's `deliveredThrough` cursor proves delivery.
 - **IC4 stretch**
 - Distinguish committed, pushed, delivered, and read as four separate states with separate observability.
- Q2: Why not order by the client's timestamp?
 - **IC3 baseline**
 - Device clocks disagree and skew; the server-assigned contiguous sequence is the only shared order.
 - **IC4 stretch**
 - Keep the client timestamp for display only; a sequence gap detection depends on contiguity that timestamps cannot provide.
- Q3: The client retries and the recipient sees two copies.
 - **IC3 baseline**
 - The retry must reuse `clientId`; the unique key returns the original sequence.
 - **IC4 stretch**
 - Alarm on dedup hit rate per client version to catch clients that mint new ids per retry.
- Q4: A user with `10,000` conversations reconnects after a month.
 - **IC3 baseline**
 - The inbox index lists only conversations with new sequences; the client pages those lazily, most-recent first.
 - **IC4 stretch**
 - Cap eager catch-up to the top `N` conversations by recency and defer the rest to on-open pulls.
- Q5: A group grows to `100,000` members.
 - **IC3 baseline**
 - Switch that conversation to fan-out on read; one timeline, per-device cursors, hint-only pushes.
 - **IC4 stretch**
 - Migrate the fan-out policy explicitly with a membership-version fence so no member misses the switch.
- Q6: A removed member keeps receiving messages for a minute.

- **IC3 baseline**
 - Delivery must check membership at push time, not from a list cached at send time.
- **IC4 stretch**
 - Make the membership version part of the outbox row and reject pushes whose version is older than the removal.
- Q7 (premise reversal): Both parties are online 95% of the time; why persist every message instead of relaying and storing only for offline users?
 - **IC3 baseline**
 - The 5% case and every crash between receive and forward lose messages, and the sender cannot know which case applied at acknowledgment time.
 - **IC4 stretch**
 - Relay-first is acceptable for ephemeral signaling; for chat the durable commit is also what gives shared order and multi-device consistency, so it is not just an offline feature.
- Q8: Make presence exact so we never push to a dead connection.
 - **IC3 baseline**
 - Exact presence would need a consensus write per heartbeat and still be stale at the moment of the push.
 - **IC4 stretch**
 - Keep presence approximate; make the push path tolerant of stale routes by always having the inbox row as the fallback.
- Validation scenarios
 - Send acknowledgment lost, identical retry.
 - Two senders in one conversation at the same instant.
 - Gateway crash with messages in flight to its connections.
 - Device offline for 30 days with 50 active conversations.
 - Member removed while a delivery worker holds a cached member list.
 - Delivery worker crash after push and before recording the attempt.
 - Receipt arriving out of order across two devices of one user.
 - Route registry outage during peak sends.
 - Conversation migrated from fan-out on write to fan-out on read.

Recall card

- "Sent means committed with a sequence; delivered means the device's cursor passed it."
- "One conversation, one contiguous sequence, allocated inside the send transaction."
- "Retries reuse the intent id; the unique key returns the original sequence."
- "Presence picks push-now or inbox-later; the outbox and cursors make either path loss-free."
- "Small groups fan out on write; large channels fan out on read."
- Wrap-up script
 - "The design is a stateless gateway fleet over a conversation-sharded durable log, with a send transaction that allocates the sequence and deduplicates the sender's intent, and delivery workers driving per-device cursors."
 - "The key invariant is that a message is committed at most once per intent and delivery is a contiguous cursor over the log, so nothing is lost when a gateway dies and nothing duplicates when a client retries."
 - "The main tradeoff is fan-out on write for small groups, which costs extra writes per recipient but makes catch-up one indexed read; large channels switch to pull."
 - "My weakest assumption is client compliance with the intent-id and contiguous-cursor contract; I would validate that with server-side cursor checks and a dedup-rate alarm before trusting any client version."

References

- [Ranked question and report provenance.](#)

- [OCI role framing](#).
- [Detailed API and schema companion](#).
- [Local first-hand report collection](#).
- [Resumable upload sibling](#): attachment bytes by reference.
- [Durable event log sibling](#): replicated append, consumer offsets, and retention mechanisms the outbox and timeline reuse.
- Reference check: 2026-09-10.
 - Capacity values, group-size thresholds, and architecture choices are proposed interview assumptions.
- [RFC 6455 The WebSocket Protocol](#): long-lived bidirectional connection; no delivery guarantee beyond the TCP stream.
- [OCI Streaming overview](#): partitioned, replayable log with consumer cursors, as an outbox and large-channel timeline substrate.
- [OCI work requests](#): 202 plus operation id pattern for control-plane membership operations.